arXiv:2605.20497v1 [cs.DC] 19 May 2026

# Hypergraph Partitioning on GPU with Distinct Incident Hyperedges and Size Constraints

Marco Ronzani, Cristina Silvano, DEIB, Politecnico di Milano, Italy

***Abstract*—Hypergraph partitioning is a recurring NP-hard problem in engineering; its efficient solution at scale hinges on parallelism. This work proposes a GPU-centric algorithm for multi-level hypergraph partitioning aimed at a specific set of problem constraints: limited size and distinct inbound hyperedges per partition. Manipulating hypergraphs requires deeply nested traversals and concurrent decision-making; our constraints impose further set operations amidst that. In turn, we design algorithms around the GPU's hierarchical parallelism and our problem's specifics. When forming partitions, we materialize the hypergraph's incidence structure and unique neighborhoods in memory to exploit set sparsity and batch node-pairing scores in shared memory. Upon refining partitions, we chain node moves into improving paths and cycles, checking their validity via cumulative set size variations reduced in parallel over moves. Thus, our dominant kernels exhibit a span linear in local hypergraph parameters. Results show an average 380× speedup and a 1.2-2.0× reduction in connectivity compared to a sequential multi-level partitioner. With minor changes, we also support k-way balanced partitioning, running 5× faster than CPU methods with a ∼5% quality loss for k=2, outperforming an existing GPU partitioner at comparable runtime, with no measurable overhead from the added constraints handling logic.**

***Index Terms*—Hypergraph partitioning, parallel algorithms, GPU acceleration, incidence constraint, size constraint.**

## I. Introduction

Hypergraph partitioning is a widespread problem throughout computer science, from VLSI to scientific and high-performance computing. Being NP-hard, practical solvers rely on heuristics that increasingly trade quality of results for time as instance size grows [1, 2]. For this reason, the efficient, massively parallel implementation of hypergraph partitioning algorithms holds the potential for time savings and performance improvements across many domains. However, due to the sparse and irregular structure of hypergraphs, such algorithms are far from trivially parallelizable [1, 3].

In this work, we develop a **GPU-parallel algorithm** for **directed hypergraph partitioning** under size and incidence constraints. Each partition is limited in the number of nodes it can contain and in the number of its distinct inbound hyperedges. The goal of partitioning is to minimize the connectivity, the total weight of cuts induced by hyperedges between partitions.

This particular formulation emerges in several settings. In mapping Spiking Neural Networks (SNNs) to neuromorphic hardware, constraints reflect the limited resources of hardware cores, while a lower connectivity reduces spike traffic and transmission costs [4, 5]. In VLSI and FPGAs, finite I/O often bounds incident connections while less communication saves time and energy [1, 6]. A notable mention is chiplets-based designs, that require partitioning modules across multiple dies with a tight interface budget [7]. Other applications are workload distribution in supercomputers under limited interconnect bandwidth [8] and the optimization of parallel algorithms, like the sparse matrix-multiply kernel [3, 9]. In many of these cases, the scale of hypergraphs involved is rapidly growing past millions of nodes and hyperedges, totaling billions of pins.

By their own nature, working with hypergraphs involves nested iterations. Nodes are incident to multiple hyperedges, each containing dozens of pins. Thus, already with neighborhood traversals – going from a node to its incident hyperedges, then to their pins – hop count quickly explodes. And several such visits form the basis for partitioning [10, 11]. Our problem settings further amplify this cost with the addition of loop nests that check bounds on the number of hyperedges entering a partition. Tracking which requires repeated set unions, intersections, and deduplication.

Although traversals themselves are trivially parallel, hypergraph partitioning algorithms do not align well with parallel hardware. For one, involved heuristics often rely on sequential decision making and backtracking, that result in highly contended atomic operations and heavy synchronization [12]. Even then, a hypergraph's incidence structure is typically irregular and hyperedges are unevenly distributed. Hence, achieving optimal workload distribution would presuppose that the hypergraph is already well partitioned [1].

To this day, graph and hypergraph partitioning on CPU has been extensively studied [1], notable sequential works being hMETIS [13], KaHyPar [14], and PaToH [15], with also multi-threaded developments such as Mt-KaHyPar [16], BiPart [17], and Zoltan [18]. Moving to GPU, efficient hypergraph-wide optimal updates have been devised to compete with complex sequential heuristics. First G-kway [19], then HyperG [12], implemented arbitrary ordering techniques to enable both parallel coarsening and refinement. Instead, gHyPart [3] explored adaptive parallelization strategies to accommodate changes in the structure of input hypergraphs. Yet, none of these approaches consider incidence constraints, nor have been tested at realistic scales beyond 10M pins [5, 7].

The algorithm we implement is based on the multi-level approach [13, 20]. In it, nodes that participate in similar sets of hyperedges are progressively clustered during a coarsening phase, until further aggregation would violate constraints. The resulting clusters define an initial partitioning, which is then refined by uncoarsening the hypergraph and evaluating node moves between partitions.

All throughout the above process, the hypergraph and its partitions must stay valid; therefore, two steps in particular are stressed by our set of constraints. Coarsening must only form valid clusters, meaning every pair of neighbors needs the intersection of their inbound hyperedges set to be computed. Refinement must land on a valid state for every partition, while still allowing violations in between improv-

ing node moves. Both such constraint checks involve tracking inbound sets inside of already deeply nested operations.

To accommodate this, we redesign the multi-level scheme around the GPU's hierarchical execution model. Every traversal and set operation is handled cooperatively by warps and threads. Data structures exploit the hypergraph's sparsity while preserving data access locality. Finally, every optimization problem is rethought such that nodes can decide independently and agree on valid actions later.

### A. Contributions

In this work, we detail a novel GPU-based multi-level deterministic hypergraph partitioner that handles constraints on partition size and distinct inbound hyperedges per partition. To the best of our knowledge, this is the first massively parallel implementation of algorithms specifically addressing such constraints. In particular, our algorithms feature:

1) a constraint-aware coarsening strategy that estimates inbound set union size inline during neighbor scoring;
2) an exact parallel dynamic programming algorithm for maximum-weight matching on the pseudo-forest induced by proposed coarsening pairs;
3) a refinement strategy that enables the simultaneous application of interfering node moves by organizing them into gain-ranked feasible paths and swap cycles;

Furthermore, to implement these ideas efficiently on GPU, we introduce:

1) a hierarchical mapping of nested hypergraph traversals onto blocks, warps, and threads, confining sequential work to local structural parameters and yielding an effective span linear in node degree;
2) the precomputation of deduplicated neighborhoods, enabling batched neighbor scoring in shared memory;
3) a sparse event-based method to validate size and inbound hyperedge constraints in parallel for all refinement node moves, using fast prefix sums and sorting primitives;

The resulting algorithm, tested on SNNs with up to 500M pins, is on average 380× faster than a sequential multi-level equivalent, while yielding a 1.2–2.0× reduction in cut cost, an improvement of up to 2.4× over existing SNN partitioners.

With minimal changes, our approach is also applicable to the $k$-way balanced partitioning problem. We thus evaluate it on augmented versions of the ISPD98 benchmark hypergraphs [21], achieving a mean 5× speedup over a SoTA multi-threaded CPU partitioner with a 5% cut-net increase on $k=2$. This yields better quality than a SoTA GPU partitioner at comparable runtime, despite the added constraints logic.

All stated contributions represent major improvements over the prototype of our partitioner presented in [22]. Namely, in-histogram inbound set size tracking, exact matching, and moves chaining into paths and cycles. As part of our results, we demonstrate the impact of all such algorithmic design choices and parameter settings over coarsening quality, refinement effectiveness, and runtime efficiency.

The remainder of this article is structured as follows. Sec. II formalizes the partitioning problem, while Sec. III introduces our variant of the multi-level scheme. Sec. IV presents the design principles underlying our implementation. Secs. V and VI then detail our parallel algorithms. Our experiments are reported throughout Sec. VII. Finally, Sec. VIII gives a few conclusive remarks.

## II. Problem Definition

### A. Hypergraph Partitioning Model

Hypergraphs (h-graphs) are a generalization of graphs where edges can connect more than two nodes, thereby becoming hyperedges (h-edges). For us, a weighted directed hypergraph $G(N, E, \omega)$ consists of a set $N$ of nodes and a set $E$ of hyperedges. Each h-edge $e \in E$ contains a subset of nodes (or pins) $e \subseteq N$, with $src(e)$ isolating the h-edge's sources and $dst(e)$ its destinations. Owing to $e$ being a set, we assume no duplicate pins nor self-cycles, $\forall e,\ src(e) \cap dst(e) = \varnothing$. The function $\omega : E \to \mathbb{R}$ assigns a weight to each h-edge. In addition, we define incidence sets $in(n) = \{e \in E \,|\, n \in dst(e)\}$ and $out(n) = \{e \in E \,|\, n \in src(e)\}$ as the sets of inbound and outbound h-edges from a node $n \in N$, and $\mathcal{I}(n) = in(n) \cup out(n)$ as the node's set of incident h-edges. Lastly, we denote the neighbors of a node $n \in N$ as $\mathcal{N}(n) = \{m \in e \,|\, e \in \mathcal{I}(n)\} \setminus \{n\}$. That being the subset of nodes partaking in any h-edge together with $n$.

A partitioning of $G$ is a set $P \subset \mathcal{P}(N)$ of pairwise disjoint subsets – partitions – of its nodes such that $\bigcup_{p \in P} p = N$. With $\mathcal{P}(\cdot)$ denoting the power set. Equivalently, a partitioning can be represented by a function $\rho : N \to P$ assigning each node to a partition, where $\rho(n) = p$ if and only if $n \in p$.

For convenience, we assume an arbitrary total order to exist over nodes $\prec_{id} N$, h-edges $\prec_{id} E$, and partitions $\prec_{id} P$, induced by their id-based representation in memory.

### B. Objective and Constraints

Our partitioning constraints pose hard limits on the number of nodes and of distinct inbound h-edges per partition. Let $\Omega$ be the maximum size of a partition, from which $\forall p \in P, |p| \leq \Omega$. Let $\Delta$ be the maximum number of allowed distinct inbound h-edges to a partition, hence $\left|\bigcup_{n \in p} in(n)\right| \leq \Delta$. Furthermore, let $pins(p, e) = |\{n \in e \,|\, \rho(n) = p\}|$ be the number of pins h-edge $e$ owns in partition $p$, and $pins_{in}(p, e) = |\{n \in dst(e) \,|\, \rho(n) = p\}|$ the number of times an h-edge $e$ is inbound to a partition $p$. The second constraint can thus be written as $\forall p \in P,\ |\{e \in E \,|\, pins_{in}(p, e) > 0\}| \leq \Delta$.

Our objective function is the **connectivity**, the number of cuts on each h-edge times its weight. That is, we pay once an h-edge's weight for every partition beyond the first it touches:

$$Conn_G(\rho) = \sum_{e \in E} \omega(e) \cdot (|\{\rho(n) \,|\, n \in e\}| - 1). \tag{1}$$

The goal for partitioning is to minimize connectivity subject to the above constraints.

While the remainder of this work focuses on the distinct inbound h-edges constraint, it must be noted that all presented solutions can be trivially reformulated to consider a distinct outbound or incident h-edges count constraint and undirected h-graphs. In addition, the present discussion assumes that a valid solution always exists.

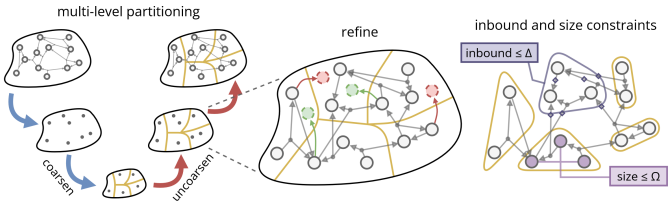



Fig. 1: Overview of the multi-level hypergraph partitioning scheme.

## III. The Multi-level Partitioning Scheme

The multi-level heuristic comprises the *coarsening*, *initial partitioning*, and *uncoarsening* phases, as shown in Fig. 1. The idea is to progressively simplify the partitioning problem by coarsening nodes into larger and larger clusters, until few enough remain that a good initial partitioning can be cheaply determined. Then, clusters are undone while localized attempts are made to improve the solution [1, 6, 13]. We here rethink this classic scheme in light of our constraints. Our coarsening routine implements edge-coarsening [6], while refinement is based on the Fiduccia–Mattheyses algorithm [20]. Constraint checks aside, the h-graph is treated as undirected.

The first coarsening level takes $G(N, E, \omega)$ and constructs its coarse version $G'(N', E', \omega')$. Where $N' \subset \mathcal{P}(N)$ is a set of coarse nodes, pairwise disjoint clusters over $N$ with $\bigcup_{n' \in N'} n' = N$. This is complemented by a node to cluster assignment function $\gamma : N \to N'$ such that $\gamma(n) = n'$ iff $n \in n'$. Coarse h-edges $E'$ are built accordingly as $E' = \{\{\gamma(n) \mid n \in e\} \mid e \in E\}$ and $\omega'(e') = \omega(e)$. To pace the process, we limit each cluster to at most two nodes $\forall n' \in N', |n'| \leq 2$. As we coarsen we track the overall size of coarse nodes as $size : N \cup N' \to \mathbb{N}^+$, where $\forall n \in N, size(n) = 1$ and $\forall n \in N', size(n') = |n'|$. Clusters form the basis for partitions and must thus respect the same constraints.

The goal for coarsening is to cluster together nodes appearing in similar sets of h-edges [10]. That is, clusters should be formed by neighbors maximizing the total weight of h-edges connecting them:

$$\begin{aligned} Score_G(\gamma) &= \sum_{e \in E} \sum_{n' \in N'} \omega(e) \cdot (|\{n \in n' \mid n \in e\}| - 1) \\ &= \sum_{e \in E} \omega(e) \cdot (|e| - |\{\gamma(n) \mid n \in e\}|). \end{aligned} \tag{2}$$

The process now repeats analogously using $G'$ as input, and onward. Running for a runtime-dependent $l$ levels in total, producing the $\gamma_1, \ldots, \gamma_l$ sequence of coarsening maps. Coarsening stops as soon as the lowest possible valid number of nodes $\lceil |N|/\Omega \rceil$ is reached. Alternatively, it stops when no further valid clusters can be built.

Coarsening itself produces our initial partitioning, with clusters on the $l$-th level coinciding with the initial partitions. This works by recognizing that the goal of coarsening, maximizing $Score_G(\gamma)$, is the dual of partitioning's minimization of $Conn_G(\gamma)$ when $\gamma$ is interpreted as defining partitions. Indeed, for fixed $G$, the terms $\sum_{e \in E} \omega(e)\,|e|$ and $\sum_{e \in E} \omega(e)$ are constant, rendering Eq. 2 and Eq. 1 equivalent objectives up to an additive constant. Moreover, with no constraints on the number of partitions, the lowest cut cost naturally occurs with an almost minimal number of partitions [4].

The initial $\rho$ can be recovered as $\rho(n) = \gamma^l(\gamma^{l-1}(\ldots \gamma^1(n)))$, by projecting partitions backward through levels. Coarsening, however, only aims to minimize connectivity on a per-level basis, implying that, as levels uncoarsen, gaps for improvement appear. Hence, every level is followed by local refinement techniques [6].

During refinement, each node is moved to a different partition if doing so happens to fully disconnect some h-edges from its current one, sparing more cuts than it creates [20]. In detail, a node $n \in N$ of a level's input h-graph is moved from its partition $p_s$ to $p_d \in P$ if:

$$\sum_{e \in \mathcal{I}(n) \text{ s.t. } pins(p_s,e)=1} \omega(e) > \sum_{e \in \mathcal{I}(n) \text{ s.t. } pins(p_d,e)=0} \omega(e) \ . \tag{3}$$

Again, only enacting moves within constraints. Refining at every level leverages the reduced graph size to propagate improvements efficiently to many original nodes.

## IV. Efficient Hypergraphs Handling on GPU

### A. The GPU Hierarchical Parallelism Model

Our present terminology hinges on CUDA, which is our API of choice for general-purpose processing on GPU. From an architectural perspective, a GPU comprises several streaming multiprocessors, each handling several threads grouped in blocks. Internally, a multiprocessor breaks a block into warps, sets of 32 threads that undergo SIMD execution. Threads have access to a limited number of registers, while blocks can allocate a small amount of shared memory, a scratchpad seen by all their threads. Any other data resides in global memory, backed by VRAM. The result is a hierarchical parallelism model, spanning blocks, warps, and threads.

For a h-graph $G(N, E, \omega)$, problem size scales with nodes $|N|$, h-edges $|E|$, and pins count $\sum_{e \in E} |e|$. In contrast, h-edge cardinality $d = \max_{e \in E}(|e|)$, node incidence degree $h = \max_{n \in N}(|\mathcal{I}(n)|)$, and partitions count $|P|$ are local, structural parameters that remain comparatively small in practical instances. Hereafter, we thus assume $|N|, |E| \gg h, d$ and parallelize first across nodes and h-edges, while confining structural iterations to inner parallelism levels.

H-graph algorithms are based on nested traversals. A flat iteration over nodes or h-edges forms the outer loop. When incidence matters, each node expands to its incident h-edges; exploring neighbors further expands each h-edge into its pins. This progresses like:

$$\underbrace{\forall n \in N}_{\text{nodes} \to} \ , \ \underbrace{\forall e \in \mathcal{I}(n)}_{\text{incident hedges} \to} \ , \ \underbrace{\forall m \in e}_{\text{neighbors}} \qquad W = O(|N| \cdot h \cdot d) \tag{4}$$

where work grows multiplicatively with structural depth.

To map these traversals on GPU, we fold them twice over the parallelism hierarchy. Outer iterations over $N$ and $E$ are distributed across blocks and warps. A first structural iteration is handled cooperatively within a warp. Whereas a second one is performed sequentially by entire warps – not by threads, thus minimizing warp divergence. Consequently, any sequential processing is restricted to the small structural dimensions $h$ or $d$ of the h-graph.

When discussing algorithmic complexity, we report it as a pair of **work** $W$ and **span** $S$. Respectively, the total amount

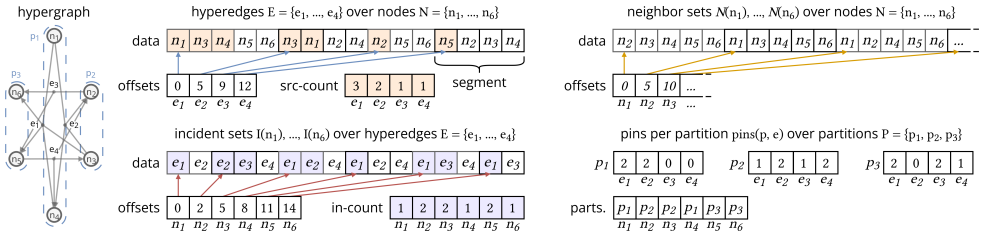



Fig. 2: Example of a hypergraph's compressed sparse memory representation.

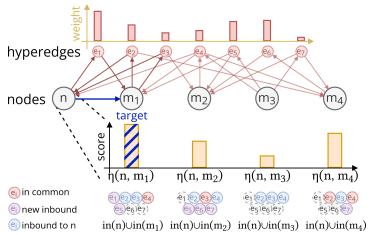



Fig. 3: Histogram over $n$'s neighbors.

of computation and the residual serial depth after mapping parallelism onto the GPU hierarchy. Work distributed across blocks, warps, and threads is treated as ideally parallel; only operations that remain serial within a lane contribute to $S$. For brevity, every complexity reported here is an asymptotic upper bound and we omit the $O(\cdot)$ notation.

### B. Hypergraph Data Structures

We store h-graphs in a compressed sparse format, with their internal structure traversed at warp granularity. See Fig. 2. This layout both mitigates warp divergence and promotes memory access coalescing.

An h-graph is primarily described by sets of sets, namely $E$ and $\mathcal{I}(n)$. The memory representation of such two-level structures in compressed sparse form involves two arrays. A segmented data array stores contiguously each linearized inner set. Then, an array of offsets maps inner set ids to their data's starting position in the previous array. If now one or few warps handle each segment, they will see fully coalesced accesses and little divergence. Nodes and h-edges alike are identified by unsigned integers, their ids constituting all atoms inside sets. With ids being a zero-based range, they double as indices in the offsets array.

To fully match the formalization in Sec. II, h-edges and incidence sets also need their entries to be separable across $src(\cdot)$-$dst(\cdot)$ and $in(\cdot)$-$out(\cdot)$ respectively. For this reason, we store in each h-edge segment all source pins first, while for incidence sets we store inbound h-edge ids first. Then, we keep a secondary offsets array for each of these data structures, containing $|src(\cdot)|$ and $|in(\cdot)|$ respectively, to enable precise access to each subset.

These data structures reside in global memory, and their compressed layout is optimized for coalesced access by design when an entire warp is used to iterate over each segment. It follows that warp shuffles can be used to trivialize most parallel patterns within a segment. All kernels described in the remainder of this work implicitly rely on this layout, even if hidden behind mathematical objects.

## V. Coarsening

### A. Algorithm Overview

Coarsening starts with the construction of mutually exclusive pairs of nodes, which is carried out in two steps. First, each node selects among its neighbors the most suitable pairing target. Every node and its target form a candidate pair for coarsening. Then, actual coarse nodes are determined by a maximum-weight matching computed over candidate pairs.

A node $n$'s pairing target is the neighbor it is connected to with the highest total weight. A target must also be valid, as in target and node must be able to form a valid cluster within constraints. Target selection involves visiting $n$'s incident h-edges and their pins, accumulating for each neighbor the total weight of h-edges it appears in. Doing so for every node costs $W = |N| \cdot h \cdot d$. The result is a histogram over the node's neighbors:

$$\forall m \in \mathcal{N}(n),\ \eta(n,m) = \textstyle\sum_{e \in \mathcal{I}(n) \text{ s.t. } m \in e} \frac{\omega(e)}{|e|} \ . \tag{5}$$

With $\eta : N \times N \to \mathbb{R}$ defaulting to zero for non-neighbors, and weights normalized by h-edge size to limit the influence of high-cardinality ones. Finding the best valid target then takes repeated maximum extractions from the histogram followed by constraint checks until a valid neighbor is found. With $target : N \rightharpoonup N$ and related $score : N \rightharpoonup \mathbb{R}$, this is:

$$\begin{aligned} &target(n) = \max_{id} \operatorname{argmax}_{m \in \mathcal{N}(n) \text{ s.t. } valid(n,m)} \ \eta(n,m) \ , \\ &score(n) = \eta(n, target(n)) \ , \\ &valid(n,m) \Leftrightarrow size(n) + size(m) \leq \Omega \ \wedge \ |in(n) \cup in(m)| \leq \Delta \ . \end{aligned} \tag{6}$$

Checking cluster size is trivial, while distinct inbound h-edges require computing the union set size between a node's inbound set and each neighbor's. Finally, *target* defines the proposed candidate pairs, weighted by the corresponding *score*.

Candidate pairs and scores form a directed weighted graph overlaying the h-graph, that we call proposal graph. Any node can partake in at most one cluster, so selecting mutually exclusive node pairs reduces to a maximum weighted matching problem over said proposal graph [10]. However, we observe that by virtue of every node proposing one candidate pair, the proposal graph is a pseudo-forest. Additionally, both neighbor histograms and validity are symmetric, i.e. $\forall n, m \in N$ it holds $\eta(n,m) = \eta(m,n)$ and $valid(n,m) \Leftrightarrow valid(m,n)$. Therefore, every edge entering a node must have score less than or equal to the edge leaving it, $\forall n, m \in N$, $target(m) = n \Rightarrow score(n) \geq score(m)$. This implies that along each component's cycle the score is constant. In particular, by the definition of *target* with $\max_{id}$, all cycles have length two. In other words, for every candidate pair, either the two nodes target each other, or one has a neighbor, not in common with the other, to which it connects with a higher score. With this structure, matching admits an exact dynamic programming solution in $W = |N|$ [23, 24].

Once final mutually exclusive pairs are determined, they become the next level's coarse nodes. Then, constructing the coarse h-graph in full involves several set unions and reconstructions. Notably, merging inbound sets between paired nodes and mapping h-edge pins from nodes to clusters. Both

operations requiring deduplication at scale to preserve their sets' nature.

### B. Materializing Neighbors

Coarsening requires a view of each node's unique neighbors to build the $\eta$ histogram over them. However, without knowing unique neighbors a priori, a direct implementation would require histogram bins to be overallocated, as they must also serve for deduplication. Each node's histogram would then demand enough memory to fit up to $d \cdot h$ neighbors, an amount that easily exceeds shared memory capacity and forces histograms to spill to global memory. Hence, histogram construction can turn into a very costly operation, requiring many random, atomic global accesses.

To minimize the cost and memory footprint of working with unique neighbors, we fully materialize $\mathcal{N}(\cdot)$ in memory once for the initial h-graph and progressively update it in-place while coarsening.

This does not fundamentally alter the asymptotic complexity of building the histogram, a full traversal of incident h-edges and pins per node is still required. However, it brings several advantages. It offsets the repeated cost of deduplication from candidate pairs proposal to a single, upfront construction. When moving down one level, coarse neighbors are computed from existing ones, progressively dealing with fewer duplicates. Deduplicating sets of neighbors alone, with no accumulated histogram weights on them, occupies exactly half the memory. As a result, the one-time overhead of initial neighbors construction is amortized over all its fast updates on subsequent coarsening levels.

Materialized neighbors too follow the compressed sparse format from Sec. IV-B. Besides, unique neighbors are solely needed during coarsening, and their memory can be reclaimed afterwards.

### C. Candidate Pairs Proposal

To construct candidate pairs, each node $n$ shall build the histogram $\eta(n, \cdot)$ over its neighbors. With $n$'s unique neighbors now known, we load a fixed-size batch of them at once such that the histogram fits in shared memory. A pass over incident h-edges and pins then fills the histogram, from which the highest weight valid neighbor is extracted. The process is repeated for all neighbor batches.

We assign a warp per node $n \in N$ and have its threads visit all pins of its incident h-edges. First, the warp collectively loads a batch of $m \in \mathcal{N}(n)$ in shared memory, then sorting it in histogram bins by id. Each bin accumulates $\eta(n, m)$, starting from zero. The warp proceeds to iterate over $\mathcal{I}(n)$, having threads read consecutive pins and incrementing their histogram entry by each h-edges's weight. For every pin, a binary search is used to find its bin, and since a node can appear at most once per h-edge, increments can be non-atomic. Subsequently, the warp sorts the histogram $\eta(n, \cdot)$, using neighbor ids as a deterministic tie-breaker. The maximum of the batch is then repeatedly extracted, and upon passing constraints checks, it can update the highest-scoring target.

Constraints checks on cluster size are easily performed by keeping track of $size(\cdot)$. Instead, for inbound set sizes, we compute $\forall m \in N, inter(n, m) = |\{e \in E \mid n, m \in dst(e)\}|$ as the size of the intersection of $n$ and $m$'s inbound sets. Knowing $inter(n, m)$, we can infer $|in(n) \cup in(m)|$ as $|in(n)| + |in(m)| - inter(n, m) \leq \Delta$, enabling immediate inbound connections constraint check. Crucially, $inter$ can be computed for almost free, since it requires a visit of every neighbor via all h-edges connecting it to $n$, exactly what is already being performed for the histogram. In each histogram bin, say of neighbor $m$, alongside $\eta(n, m)$, we also keep a counter representing $inter(n, m)$, initially zero. With pins of an h-edge being unique by definition, every time a neighbor is seen as the destination – $m \in dst(e)$ – of an h-edge in $e \in in(n)$, its counter increments. As a result, constraint checks introduce minimal overhead to candidate pairs proposal. See Fig. 3 for an example. The final kernel's span is $S = h$.

To improve candidates quality and ensure a steady coarsening process, we augment the above histogram kernel with three mechanisms.

As the h-graph is coarsened, it often occurs that a group of multiple nodes partakes in exactly the same h-edges, thus all achieving the same score across their histograms. With ties broken by id, every node will thus target the same lowest-id neighbor. But this is undesirable, since all nodes end up forming a star around a single candidate pair, wasting all other equally high-score connections they shared. In response, we introduce some deterministic noise to subtly diversify scores. Each histogram bin is given a small pseudo-random value: $\eta(n, m) = \cdots + rng(min(n, m), max(n, m))$. With the noise being symmetric and conditioned on both nodes, histogram symmetry is preserved. Noise caps at 10% of mean h-edge weight.

A further optimization permanently purges invalid neighbors from their sets. When checking candidate constraints, a dedicated flag is set for each invalid neighbor. During coarse h-graph construction, flags are preserved across set merges, marking all occurrences of the same neighbor. Before finalizing the new coarse sets, flagged entities are filtered out.

At last, to ensure that coarsening always reaches close to the limit of constraints, we attempt a best-effort pairing of nodes that are left with no neighbors. After the above proposal completes, all such nodes are gathered and sorted by size. Each node, handled by a thread, then runs a binary search for its current cluster size slack and tries to atomically claim the first valid node it finds. Contentions are broken by id. At this stage, inbound set union sizes are overestimated with the sum of each node's inbound set size.

### D. Maximum Weight Matching

Now that we have the proposal graph, we must isolate a subset of mutually exclusive pairs of highest total score. This is a weighted matching problem over the pseudo-forest of candidate pairs. Given the proposal graph's structure discussed in Sec. V-A, we propose the following dynamic programming formulation of the problem.

For convenience, let us define $child : N \to \mathcal{P}(N)$ as $child(n) = \{c \in N \mid target(c) = n\}$, giving the set of nodes targeting another. From it, the proposal graph's invariant can be rewritten as $\forall n \in N, \forall c \in child(n), score(n) \geq score(c)$.

The basic problem addressed by this formulation is the maximum-weight matching over a subtree rooted in an arbitrary node $n \in N$. To solve it, we introduce two values associated with each node:

- $ss_0(n)$: maximum score of a matching in the subtree rooted in $n$ when $n$ is not matched to $target(n)$;
- $ss_1(n)$: maximum score of a matching in the subtree rooted in $n$ when $n$ is matched to $target(n)$;

For the ensuing discussion, let us isolate root nodes of the proposal graph as $R = \{n \in N \mid target(target(n)) = n\}$, thus avoiding circular dependencies in the recursion. Then, for every $n \in N \setminus R$, the second value can be expressed as:

$$ss_1(n) = score(n) + \textstyle\sum_{c \in child(n) \setminus R} ss_0(c) \ . \tag{7}$$

While on root nodes $n \in R$:

$$ss_1(n) = score(n) + \textstyle\sum_{c \in (child(n) \cup child(target(n))) \setminus R} ss_0(c) \ . \tag{8}$$

When node $n$ matches with its children $c'$, the first value is:

$$ss_0(n) = \textstyle\sum_{c \in child(n) \setminus R,\, c \neq c'} ss_0(c) + ss_1(c') \ . \tag{9}$$

Thus, let $ss_{1-0}(n) = ss_1(n) - ss_0(n)$ be the return for any node $n \in N$ matching with its target. In general, the first value will depend on a node's highest $ss_{1-0}$ children, that is:

$$ss_0(n) = \textstyle\sum_{c \in child(n) \setminus R} ss_0(c) + \max\left(0,\ \max_{c \in child(n) \setminus R} ss_{1-0}(c)\right) \ . \tag{10}$$

Let $match : N \rightharpoonup N$ define the final matching partner for each node, being undefined if the node is unmatched. We infer $match$ from $ss_0$ and $ss_1$, starting from the case of roots $n \in R$, as:

$$match(n) = \begin{cases} target(n) & \text{if } ss_1(n) > ss_0(n) + ss_0(target(n)) \ , \\ \underset{c \in child(n)}{\operatorname{argmax}}\ ss_{1-0}(c) & \text{otherwise} \ . \end{cases} \tag{11}$$

Finally, for every other node $n \in N \setminus R$:

$$match(n) = \begin{cases} target(n) & \text{if } match(target(n)) = n \ , \\ \underset{c \in child(n)}{\operatorname{argmax}}\ ss_{1-0}(c) & \text{if } match(target(n)) \neq n \wedge \\ & \max_{c \in child(n)} ss_{1-0}(c) > 0 \ , \\ \text{undefined} & \text{otherwise} \ . \end{cases} \tag{12}$$

This formulation exhibits optimal substructure: the optimal matching for the subtree rooted at $n$ depends exclusively on the optimal matchings of its children's subtrees.

Both values $ss_0$ and $ss_1$ can be computed for all nodes by a single bottom-up traversal of each pseudo-tree. At branching nodes, contributions from all children accumulate, and the locally optimal child choice is refined whenever a higher-scoring subtree is encountered. Once all upward computations are complete, a top-down traversal of each connected component can finalize the matching.

The aforementioned bottom-up and top-down traversals can be realized in parallel by launching one thread for every leaf in the pseudo-forest. Each thread then moves upward along the path $n$, $target(n)$, $target(target(n))$, ..., amending every $ss_0(n)$ and $ss_1(n)$ with the scores of the subtree it has traversed so far, see Fig. 4. To support the upward walk, we provisionally build $match$ asymmetrically: $match(n)$ is continuously updated to point to the best current $child(n)$ and

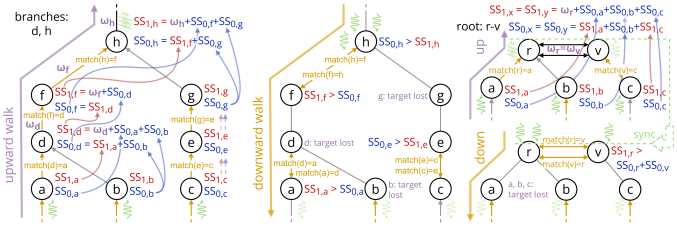



Fig. 4: Dynamic programming formulation for maximum weighted matching in a two-cycle pseudo-forest.

tracked as part of a tuple $(ss_{1-0}(match(n)), match(n))$. Before moving upward, a thread on node $n$ tries to use $ss_{1-0}(n)$ to claim $match(target(n))$. Claiming occurs via an atomic lexicographic max over the target's tuple with $(ss_{1-0}(n), n)$. This is consistent with the tie-breaking used for candidate pairs, ensuring the deterministic convergence of $match$. Continuing, $ss_0(n)$ is atomically added to $ss_1(target(n))$, and $ss_0(target(n))$ is recomputed based on the claim's outcome.

Once every thread reaches a root, they all synchronize, settle the matching for the roots, and the downward walk begins. Each thread retraces its full path backwards, checking if its claims were successful, and if so, permanently constructing matches. A node $n$ is matched with its target if, at that point, it still holds $match(target(n)) = n$. If so, $n$ discards whichever claims it received and imposes $match(n) = target(n)$. Otherwise, the same reasoning is repeated one step down the line: $n$ remains unmatched to its target and will end up matched with the current $match(n)$, i.e. the highest $ss_1$ holder among $child(n)$.

While some threads may partially walk the same path, that doesn't constitute a problem, as they will take the same decisions. Rather, to finalize the exclusive part of its path, each thread must know everything that happened between its root and leaf, therefore some redundant work is inevitable to run the descent in parallel. This routine's span is the maximum height of a tree, that typically being a very small value, we treat as a constant, hence $S = 1$.

To increase the likelihood of nodes forming clusters, candidate pairs proposal (Sec. V-C) actually produces $\Pi$ candidates, in order of decreasing score. These correspond to the top-$\Pi$ valid neighbors per histogram, leading to $\Pi$ proposal graphs. Matching then repeats for $\Pi$ rounds; $\Pi = 4$ by default. To preserve the invariant of each proposal graph, nodes matched in earlier rounds are removed from all subsequent graphs. Any instance of $target$ pointing towards one such node thus reverts to being undefined. Ultimately, the combination of deterministic noise and multiple candidates ensures that almost every node finds a pair.

### E. Coarse Hypergraph Construction

With $match$ defining clusters, coarsening finishes by constructing the coarse h-graph $G'(N', E', \omega')$ as per Sec. III. For that, the nodes-to-coarse-nodes map $\gamma$ is built as $\forall n \in N, \gamma(n) = \{n, match(n)\}$ if $match(n)$ exists and $match(match(n)) = n$ else $\gamma(n) = \{n\}$. Then, constructing coarse instances of all two-level structures, $E$, $\mathcal{I}(\cdot)$, $\mathcal{N}(\cdot)$ requires a sequence of map patterns through $\gamma$ and set unions.

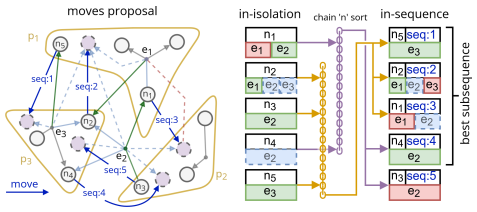



Fig. 5: Moves sequence-of-chains construction.

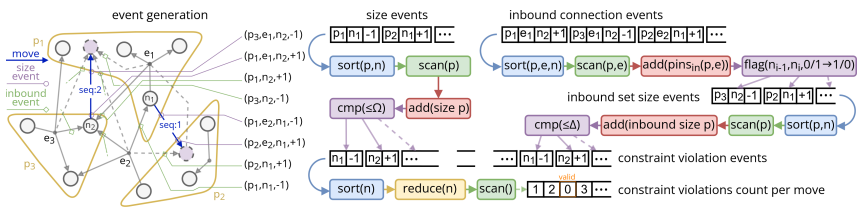



Fig. 6: Events-based moves validity check.

Both operations involve deduplication and the final set size isn't known a priori, therefore a compressed representation can't be built directly. Constructing many sets in parallel requires two phases: first, an oversized data array is built and deduplication takes place over it, then, its segments are packed in the final compressed form.

Each coarse set is assigned to a block and given an oversized global memory segment to work with. The block is also given the maximum possible amount of shared memory; both memories are configured as closed-hashing hash-sets. Threads in the block collectively read elements to insert in their set, using $\gamma$ as needed to map nodes to clusters before inserting them. Insertion of each element is first attempted in shared memory, going to global memory only upon a successful insertion or exhausted probe-length. Once all elements have been seen, the content of shared memory is cooperatively moved to the remaining holes in global memory. While this unfolds, each set's size is tracked, with a subsequent prefix-sum of set sizes providing both the final compressed data array size and the offset for each set. Lastly, a packing operation scatters each oversized array segment into its final segment in the newly allocated compressed data array.

The major remaining issue is knowing how large of an allocation is needed for each oversized array segment. H-edges do not involve any set union, hence each current segment size is a viable upper bound. For neighbors, instead, each new segment size must be the sum of the sizes of segments that will be merged into it. Finally, for incidence sets there is a shortcut: with h-edges coarsened first, count how many times each coarse node occurs as a new pin; that count is the number of distinct h-edges incident to every coarse node.

Note that to ensure the correctness of inbound sets while guaranteeing no self-cycles, eventual duplicates between $src(\cdot)$ and $dst(\cdot)$ or $in(\cdot)$ and $out(\cdot)$ are discarded from $src(\cdot)$ and $out(\cdot)$, respectively.

## VI. Uncoarsening and Refinement

### A. Algorithm Overview

Local refinement improves a partitioning by selectively moving nodes across partitions. Choosing suitable moves takes two steps. First, each node independently selects the partition it would rather belong to, proposing a move. Then, a subset of moves is found such that, when applied together, they lead to a valid state of lowest possible connectivity.

Each node $n$ proposes its move in-isolation, only with regard to the current partitioning. By Eq. 1, a cut is only avoided when an h-edge has no pins left in a partition. So, a favorable move is one that fully disconnects h-edges from $n$'s current partition while introducing cheaper connections, if any, to its new partition, as per Eq. 3. To find such moves, a node must count the number of pins each of its incident h-edges owns in every partition, what we defined as $pins(p, e)$ in Sec. II-B. Leaving the current partition spares the weight of h-edges with only a single pin, the node, remaining in it. Entering another partition costs the total weight of h-edges that currently hold no pins in it. The difference between two said quantities is the gain in connectivity for a move:

$$\begin{aligned} saving(n) &= \textstyle\sum_{e \in \mathcal{I}(n) \text{ s.t. } pins(\rho(n),e)=1} \omega(e) \ , \\ loss(n,p) &= \textstyle\sum_{e \in \mathcal{I}(n) \text{ s.t. } pins(p,e)=0} \omega(e) \ , \\ gain(n,p) &= saving(n) - loss(n,p) \ . \end{aligned} \tag{13}$$

Moving node $n$ to partition $p$ is favorable if $gain(n,p) > 0$. In fact, Eq. 13 is a direct adaptation of Eq. 3. The move with highest gain is proposed by the node as $move : N \to P$, $move(n) = \max_{id} \operatorname{argmax}_{p \in P} gain(n,p)$. Accordingly, let $p_s^n \xrightarrow{n} p_d^n$ denote a move of node $n$ from partition $p_s^n = \rho(n)$ to $p_d^n = move(n)$. Ultimately, computing gains calls for another neighborhood traversal, with $W = |N| \cdot h \cdot d + |N| \cdot |P|$.

So-obtained moves have been proposed separately, but to maximize gain, several of them shall be applied at once. Deciding which moves to apply equates to finding the subset of moves collectively leading to a valid maximum gain partitioning. However, moves easily interfere, influencing each other's gain and feasibility, making this a problem only solvable in exponential time. For this reason, we revise a heuristic from [12]. We sort moves into a sequence and only apply a subsequence of them from the first one onward. In building the sequence, we assemble chains of moves with source-to-destination concatenation and involving nodes of similar size. Chains are then sorted by total gain to form a sequence that promotes feasible and favorable swaps and cyclic exchanges.

With moves ordered, the problem of deciding which moves to apply reduces to finding the longest subsequence of improving moves landing on a valid state. To solve it, each move recomputes its gain in-sequence, i.e. assuming all moves before it already applied. Analogously, validity over constraints is computed as of every move along the sequence. A filtered maximum extraction then leads to the desired subsequence, whose moves are applied. The whole refinement process repeats $\Theta$ times per level; $\Theta = 16$ by default.

### B. Refinement Moves Proposal

Proposing moves in-isolation starts with each node computing gains over partitions. Every warp handles a node $n$, currently in partition $p_s$. First, it allocates one variable for every partition in shared memory, representing $saving(n) = 0$ and $\forall p \in P \setminus \{p_s\}$, $loss(n,p) = 0$. Then, threads iterate $n$'s in-

cident h-edges. For every h-edge, if $pins(p_s, e) = 1$, $saving(n)$ increments by $\omega(e)$, and for every partition $p_d \neq p_s$, if $pins(p_d, e) = 0$ , $loss(n, p_d)$ increments by $\omega(e)$. A map-reduce operation computes each *gain* from *loss* and *saving*, finds the maximum, and yields the node's proposed move.

Gains exclusively depend on the count of h-edge pins per partition, $pins(\cdot, \cdot)$. With the same values of *pins* reused multiple times across nodes, they shall be precomputed in parallel [12]. Thus, we prepare a matrix $|P| \times |E|$ with the values of $pins(p, e)$ before commencing refinement, see Fig. 2. Every warp is assigned an h-edge and allocates one counter in shared memory per partition. Its threads then iterate over the h-edge's pins and atomically increment counters after applying $\rho$ to every read pin. Overall, this yields a span $S = 1$.

At this stage, the first half of refinement repetitions can propose moves that invalidate a partition by size. The second half, instead, imposes $size(n) + \sum_{m \in p_d} size(m) \leq \Omega$ to focus on smaller, consistent improvements. In any case, the next steps strictly enforce final validity.

### C. Moves Sequence Construction

The efficacy of refinement is strongly limited by the joint feasibility of multiple moves. For several moves to consistently return to a valid state, they shall realize swaps or cyclic exchanges between nodes with compatible effects on the constraints of their partitions. Thus, the goal for sequence construction is to produce a total ordering of moves that consists of several chains – paths or cycles – preferably of length two or more. A chain concatenates moves each departing from the destination of the previous one, with minimal size and inbound set variation between their affected nodes. Arranging chains by decreasing internal total gain gives the final sequence. Formally, this is a weighted path covering problem that we solve greedily.

Moves are first sorted by the ordered pair $(p_s^n, -gain(n, p_d^n))$, then chains are constructed over several rounds. Each move is assigned a predecessor, initially empty. In a round, each free end of a chain (initially every move), consider it node $n$'s move, sifts a window of candidate successors $m \in N$ among moves with $p_s^m = p_d^n$, selecting the one maximizing a compatibility grade $gain(m, p_d^m) - \alpha \cdot |size(n) - size(m)| - \beta \cdot ||in(n)| - |in(m)||$; for us $\alpha = 10^{-6}$, $\beta = 10^{-7}$, and window size is 256. Conflicts – multiple chains choosing the same successor – are resolved by parallel atomic maximization on the grade and moved node id. Moves that secured a successor are frozen, and the process repeats up to 16 times or until no loose ends remain. See an example in Fig. 5.

After all rounds complete, the successor–predecessor relation will have induced disjoint paths and cycles. Subsequences are extracted by following predecessor links, simultaneously computing their total gain. Chains are ranked by decreasing total gain and concatenated into the final sequence, that by the one-to-one relation between nodes and moves we formalize as a total order of nodes $\prec_{seq} N$.

With moves sorted by their in-isolation gain, their in-sequence gain must be inferred. For every node $n$ with move $p_s^n \xrightarrow{n} p_d^n$, a warp iterates over $n$'s incident h-edges and their pins. For every h-edge $e \in \mathcal{I}(n)$, consider only pins $m \in e$ whose moves precede $n$'s in the sequence, $m <_{seq} n$, and related moves $p_s^m \xrightarrow{m} p_d^m$. Then, two conditions may arise on node $n$ after seeing all such $m$-s:

$$\begin{aligned} &\text{if } \underbrace{\left|\{m \mid p_d^n = p_s^m\}\right|}_{m \text{ leaving } p_d^n} - \underbrace{\left|\{m \mid p_d^n = p_d^m\}\right|}_{m \text{ also entering } p_d^n} = pins(p_d^n, e) > 0 \\ &\text{or } \exists m \text{ s.t. } p_s^n = p_d^m \text{ and } pins(p_s^n, e) = 1 \\ &\text{then } gain(n, p_d^n) \ -= \omega(e) \end{aligned} \tag{14}$$

$$\begin{aligned} &\text{if } \underbrace{\left|\{m \mid p_s^n = p_s^m\}\right|}_{m \text{ also leaving } p_s^n} - \underbrace{\left|\{m \mid p_s^n = p_d^m\}\right|}_{m \text{ entering } p_s^n} = pins(p_s^n, e) - 1 > 0 \\ &\text{or } \exists m \text{ s.t. } p_d^n = p_d^m \text{ and } pins(p_d^n, e) = 0 \\ &\text{then } gain(n, p_d^n) \ += \omega(e) \end{aligned} \tag{15}$$

In the first case, h-edge $e$, which was not cut in isolation, becomes cut in the sequence, either due to $n$ now being the first to re-enter an earlier disconnected partition, or it no longer being the last node of $e$ to leave its current partition. Conversely, in the second case, an additional cut on h-edge $e$ is spared because another node $m$ entered $n$'s destination for the first time before $n$, or $n$ suddenly became the last node of $e$ in its partition. Again, by the neighborhood traversal, $S = h$.

A sequence of nodes, chained and ranked by in-isolation gain, and carrying their in-sequence gain, is thus available.

### D. Events-based Constraint Checks

Simultaneously assessing the partitioning's validity at every move in the sequence requires reconstructing every intermediate state of all partition sizes and, critically, inbound sets. Materializing all such states is prohibitive; consequently, we handle constraint checks sparsely through "events". First, each move generates events for every partition size and inbound set size variation it causes, carrying the variation's delta as payload. Next, with a series of parallel patterns over deltas, we infer each move's validity, see Fig. 6.

Crucially, a move alters partition size for exactly two partitions, and affects inbound set sizes only through transitions of $pins_{in}(p, e)$ across zero. Exploiting this structure, every move of a node $n$, $p_s^n \xrightarrow{n} p_d^n$, generates two size-event 3-tuples $(p_s^n, n, -size(n))$, $(p_d^n, n, +size(n))$, along with several inbound connection events given by the 4-tuples $\forall e \in in(n)$, $(p_s^n, e, n, -1)$, $(p_d^n, e, n, +1)$.

Each size event $(p, n, \delta)$ means that partition $p$'s size changes by $\delta$ with $n$'s move. After being sorted by $(p, n_{seq})$, their deltas are prefix summed for each $p$; thus, each event stores its partition's cumulative size variation up to its move along the sequence.

For the inbound h-edges count, we here rely on $pins_{in}(p, e)$, rather than a set-based formulation, as shown in Sec. II-B. Therefore, we modify the precomputed *pins* matrix to represent $pins_{in}$ with a quick pass that subtracts the count of outbound h-edges.

Each inbound set event $(p, e, n, \delta)$ means that $pins_{in}(p, e)$ changed by $\delta$ after $n$'s move. They are first sorted using $(p, e, n_{seq})$ and prefix summed using $(p, e)$ as keys. To track inbound set size changes, $pins_{in}(p, e)$ is added to each respective delta, giving $e$'s running inbound pin count on

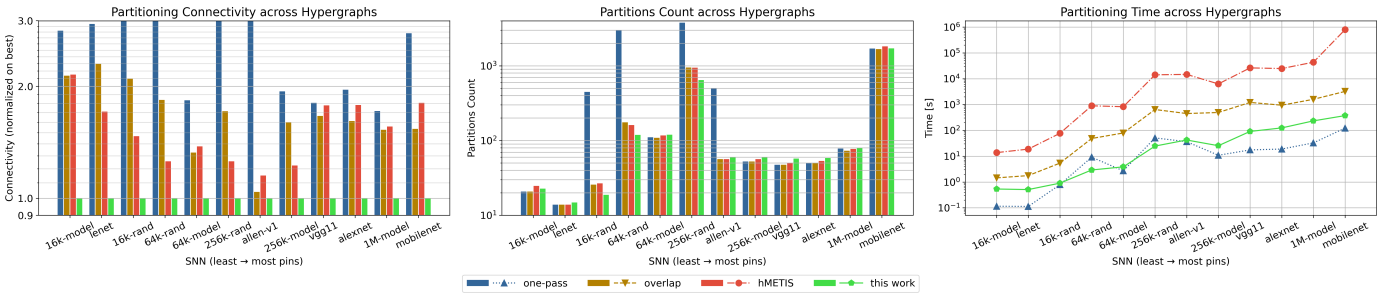



Fig. 7: Partitioning results comparison with three sequential methods across twelve SNNs hypergraphs.

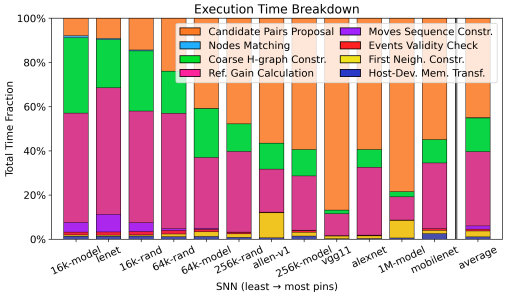



Fig. 8: Algorithm steps execution time breakdown.

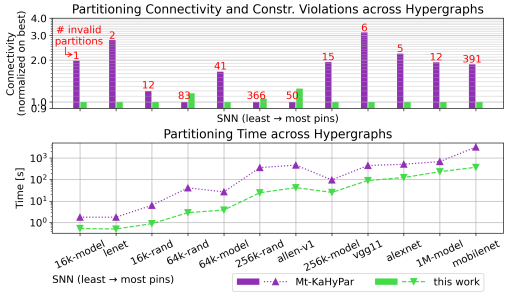



Fig. 9: Partitioning results vs Mt-KaHyPar [16].

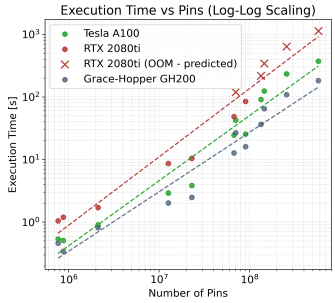



Fig. 10: Scalability across GPUs.

| | 16k-model | 64k-model | 256k-model | 1M-model | lenet | alexnet | vgg11 | mobilenet | allen v1 | 16k-rand | 64k-rand | 256k-rand |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\|N\|$ | 20k | 110k | 216k | 302k | 14k | 208k | 194k | 6.9M | 231k | 16k | 64k | 256k |
| $\sum_{e\in E}\|e\|$ | 766k | 23M | 90M | 256M | 875k | 145M | 133M | 577M | 70M | 2.1M | 12.6M | 67.4M |
| $\text{avg}_{e\in E}\|e\|$ | 37.3 | 210.3 | 417.2 | 848.1 | 63.2 | 696.2 | 688.3 | 83.5 | 304.7 | 128 | 192 | 256 |
| $\Omega, \Delta$ | $2^{10},2^{12}$ | $2^{10},2^{12}$ | $2^{12},2^{16}$ | $2^{12},2^{16}$ | $2^{10},2^{12}$ | $2^{12},2^{16}$ | $2^{12},2^{16}$ | $2^{12},2^{16}$ | $2^{12},2^{16}$ | $2^{10},2^{12}$ | $2^{10},2^{12}$ | $2^{10},2^{12}$ |

TABLE I: Spiking neural networks used in the experiments [25].

$p$. A new event $(p, n, -1)$ is then emitted whenever such counts transition from 1 on the event before to 0, or $(p, n, +1)$ when turning from 0 to 1. Resulting events are again sorted by key $(p, n_{seq})$ and prefix summed per $p$, giving each $p$'s distinct inbound h-edges count variation as of $n$'s move in the sequence.

With all events sorted by $(p, n_{seq})$, adding initial set sizes $|p|$ and $|\{e \in E \,|\, pins_{in}(p, e) > 0\}|$ respectively yields used constraint capacities move by move. A subsequent comparison with $\Omega$ or $\Delta$ shows if $p$ is valid after moving $n$. Then, looking at pairs of events for consecutive moves of $m <_{seq} n$ in the sequence tells if moving $n$ was responsible for invalidating or re-validating partition $p$ based on how it was left by $m$. This spawns a final sequence of events for every time a partition changes to invalid $(n, +1)$ or valid $(n, -1)$. When sorted and reduced by $n_{seq}$, their prefix sum is the count of active constraint violations after each move. Only moves with count zero are valid, finding the one of maximum cumulative gain gives the subsequence of moves to apply.

## VII. Experimental Results

### A. Experimental Setup

To test our solution, we conduct two types of experiments. An application-driven evaluation under our constraints, followed by validation under standard benchmarks and metrics.

First, we partition 12 h-graphs originating from SNNs and their mapping constraints on neuromorphic hardware [25], see Tab. I, chosen for their diversity in size and topology. Networks rapidly grow in size, from 1M to over 500M pins. Topologies, meanwhile, vary from a mostly local and regular

| Algorithm Step | Work | Span |
|---|---|---|
| Candidate Pairs Proposal | $\|N\| \cdot h \cdot d$ | $h$ |
| Nodes Matching | $\|N\|$ | 1 |
| Coarse H-graph Construction | $\|N\| \cdot h \cdot d + \|E\| \cdot d$ | 1 |
| Refinement Gain Calculation | $\|N\| \cdot h \cdot d + \|N\| \cdot \|P\|$ | $h$ |
| Moves Sequence Construction | $\|N\| \cdot \log \|N\|$ | $\log \|N\|$ |
| Events Validity Check | $\|N\| \cdot \log \|N\| \cdot h$ | $\log \|N\|$ |
| First Neighbors Construction | $\|N\| \cdot h \cdot d$ | $h$ |

TABLE II: Summary of work and span of every algorithm step.

structure on `-model` networks, to a small-world, erratic structure on `-rand` ones. Hence, the latter exhibit significantly larger neighborhoods. In particular, we present detailed performance metrics for two h-graphs, the `256k-model`, a VGG-like ANN [26] converted to SNN, and the `allen-v1` [27] neural model. Together, they are representative of both extremes in h-graph topologies, from regular to highly not so, and all our other benchmarks follow either of their trends.

Our baseline comprises three heuristics running sequentially on CPU. An implementation of the multi-level scheme in hMETIS adapted to our constraints [4, 13]. And two simple algorithms originating from SNN mapping tools. A greedy "overlap" [4] heuristic that co-locates nodes based on their incidence sets' overlap. And a trivial "one-pass" [5] algorithm that fills one partition after the other during a single pass over nodes, driven solely by constraints. In addition, we compare against the SoTA Mt-KaHyPar [16] multi-threaded CPU partitioner. However, Mt-KaHyPar lacks support for incidence constraints, therefore serving as a qualitative reference.

Using the same setup, we also perform a set of ablation studies to measure the impact of our design choices, namely the materialization of neighbors, optimal matching, and refinement moves chaining. We also study the effect of varying our parameters for the number of coarsening candidates ($\Pi$) and refinement repetitions ($\Theta$). Note that, aside from these experiments, every other result is based on their default values of 4 and 16, respectively.

Second, we evaluate our solution on $k$-way balanced par-

titioning, the de facto reference for h-graph partitioning algorithms [1, 2]. To do so fairly, we use the standard **cut-net** quality metric, defined as the total weight of cut h-edges:

$$Cut\text{-}net_G(\rho) = \sum_{e \in E} \omega(e) \cdot \mathbb{1}(|\{\rho(n) \mid n \in e\}| > 1). \quad (16)$$

As for h-graphs, we use the ISPD98 benchmark [21], but given the limited size of its entries, to demonstrate the scalability of massive parallelism, we randomly augment by 16× each h-graph's nodes and pins, see Tab. III. Here we compare against the multi-threaded CPU partitioner Mt-KaHyPar [16] and the GPU partitioner gHyPart [3]; we do not include HyperG [12] as no reproducible artifact is available to us.

For our implementation to abide by the $k$-way constraints, we set $\Omega = (1+\epsilon) \cdot |N|/k$ and $\Delta = +\infty$, where $\epsilon$ is a given balance parameter. Then, to ensure the construction of exactly $k$ initial valid partitions, we halt coarsening upon reaching less than 4096 coarse nodes (empirically stable for small $k$-s) and rely on Mt-KaHyPar's direct $k$-way configuration [16] to provide a robust initial partitioning. Such CPU-side work takes tens of milliseconds and is included in the end-to-end timings. In our case, the cut-net is not optimized directly, but results from minimizing connectivity.

Experiments were performed on an `A100-SXM4-40GB` GPU and `EPYC 7453 @ 2.75GHz` CPU with 256GB of RAM, CUDA version `12.4`, and GPU driver version `550.78`. Every multi-threaded execution was assigned 16 threads. All timing results are end-to-end and have been averaged over 10 runs with negligible variance.

### B. Comparison Results

SNN partitioning results are reported in Fig. 7. Our implementation achieves a mean speedup of 380× over hMETIS and 12.5× over the overlap method, remaining within just $1 \pm 0.8$× of the one-pass method. Quality of results always improves, with our solution's connectivity on average 0.64× that of hMETIS; an advantage attributable to the larger space of refinement moves explored. Follow larger improvements of 0.58× and 0.08× over the overlap and one-pass methods. The number of partitions built presents instead minimal differences, attesting to our effective handling of all constraints.

Results are consistent across all h-graphs and their topologies. In particular, sequential execution time exhibits a slight step increase on the less regular h-graphs (e.g. `256k-rand`, `allen-v1`), due to the simultaneous increase in node count and neighbors count per node. By contrast, our approach's complete parallelism over the h-graph's size, coupled with the materialization and early deduplication of neighborhoods, lead to a smoother execution time trend.

In Fig. 9 we study our solution against Mt-KaHyPar, with the latter allowed to violate the distinct inbound h-edges constraint. We overall achieve 0.56× Mt-KaHyPar's connectivity, with a speedup ranging from 2.8× to 14.7×. Observably, on three h-graphs Mt-KaHyPar manages to reach down to 0.79× our connectivity, but at the price of tens of invalid partitions. On the five largest h-graphs, notably, the gap in execution time remains between 3-9×, however, Mt-KaHyPar's connectivity is repeatedly ∼2.0× higher than ours.

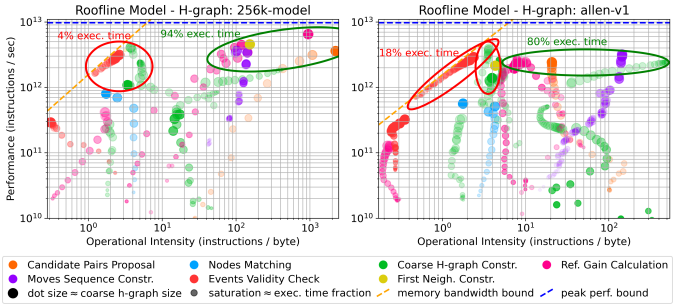



Fig. 11: Integer roofline model for all kernel launches on two SNNs. Each dot is a kernel launch: its size encodes the h-graph size, its opacity reflects the fraction of execution time within its category.

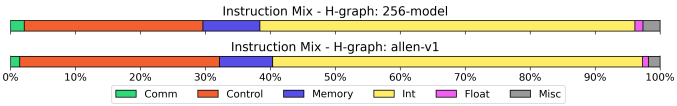



Fig. 12: Instruction mix, aggregated over a complete run.

We can thus conclude that our approach is capable of consistently achieving SoTA results under the stated constraints, both in terms of quality and time-to-solution.

### C. Performance and Scalability Analysis

In Tab. II we summarize our implementation's complexity analysis. Such bounds agree with experimental data, as both in Fig. 7, and later Fig. 15, execution time grows linearly with average h-edge cardinality so long as pins are within tens of millions (e.g. until the 64k-model). This regime is dominated by per-node or per-h-edge work. Then, as h-graph size exceeds the GPU's parallelism capacity, additional iterations are serialized within each block and execution time trends linearly with the number of pins. We note that such a linear growth matches with other GPU implementations [3, 12, 19], hinting at the absence of meaningful overheads from our additional constraints-handling logic.

To quantify the impact of each algorithm step on performance, Fig. 8 shows how execution time is distributed among them. As expected, we notice that our two neighborhood iterations, candidates proposal and gain computations, tend to dominate execution on larger inputs [10]. The latter to a lesser extent, thanks to the precomputation of *pins*. Instead, constraint checks and matching become progressively negligible, as do set operations during coarsening, confirming their ultimate inexpensiveness. Lastly, the initial construction of neighbors starts off relatively cheap, but as h-edges grow larger, non-unique neighbors grow with $h \cdot d$, making its cost noticeable, but still limited to a one-time overhead.

In our design, host-device memory transfers happen only twice, to load the original h-graph, and to bring back the finished partitioning. Initial h-edges and incidence data structures are prepared on the host while importing the h-graph and copied as is, while neighborhoods and coarse h-graphs live solely in device memory. Hence, host-device data movements take up an inconsequential amount of time.

According to the instruction mix in Fig. 12, the majority of our workload is constituted by integer (`32bit`) and control instructions, followed by memory accesses. Floating point operations are a minority, confined to pairing scores

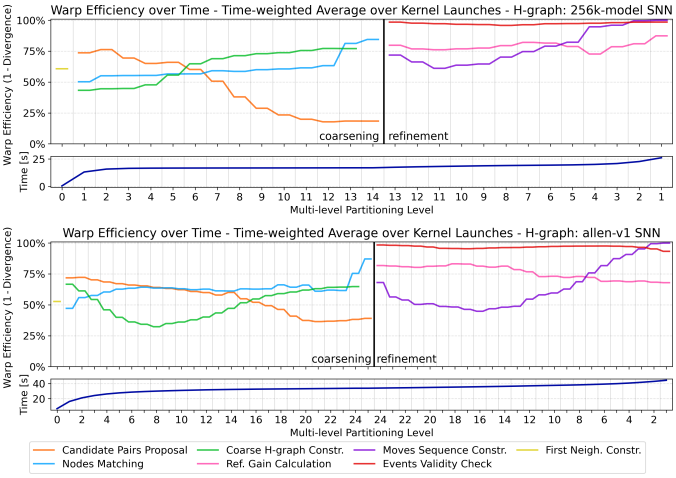



Fig. 13: Average warp efficiency over time on two SNNs.

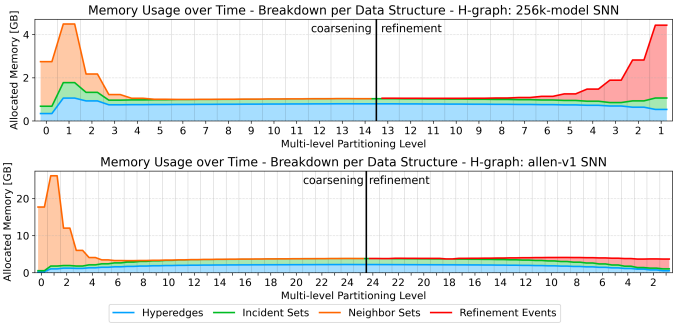



Fig. 14: Memory usage over time on two SNNs.

and move gains. This is a natural consequence of h-edges, incidence sets, and neighborhood iterations, whose nested decision-making forms the core of our algorithms.

To assess GPU resource utilization in our design, Fig. 11 presents a roofline model, where each dot corresponds to a kernel launch. Given our instruction mix, we report results in terms of integer instruction throughput, using DRAM traffic to derive operational intensity. The intricate nature of h-graphs and the resulting control flow make it challenging to reach peak performance consistently. Nonetheless, the vast majority of our global execution time, $\sim$ 90%, is spent in kernels achieving a substantial fraction of peak throughput, approximately 20–60%. This is consistent with the fact that only $\sim$ 60% of our instructions are integer and directly contribute to the measured throughput. Low-efficiency kernel launches only amount to a small fraction of runtime and occur on the coarsest levels, where nodes are unavoidably too scarce to reach significant occupancy.

Kernels predominantly operate in a compute-intensive regime, implying that our handling of costly neighborhood iterations exhibits no dominant bottleneck. Rather, the major factor influencing execution time is just the sheer number of neighbors themselves, whose processing sustains a high level of GPU utilization. The sole memory-bound steps are the parallel patterns involved in refinement events processing and a few set union operations. These take up most of the remaining execution time while still reaching non-trivial throughput, typically $>10\%$ of peak.

Interestingly, based on the roofline model, the initial neighbors construction (in yellow) should be mostly compute-bound by its large number of iterations, as seen for the `256k-model`. Yet, it approaches a memory-bound regime when the topology is more irregular, as observed on the `allen-v1`. This can be attributed to a growing number of distinct neighbors and a decrease in duplicates, leading to more updates hitting the global memory hash set.

Warp efficiency (1 – divergence), the average fraction of threads participating in the same instruction, is an indicator of proper parallelism exploitation. We report it in Fig. 13. Most of our kernels use a warp-centric mapping to handle sequential work and enable warp collectives in shared memory (e.g. histogram). On the initial, large h-graph, this sustains high intra-warp utilization, $\sim$ 70% efficiency for the bulk of runtime. On inner coarsening levels, the per-node work and neighbor lists shrink, and SIMD efficiency drops below 50%; however, these phases contribute to less than 5% of total runtime, as instances are small and finish quickly. Refinement shows instead a more uniform, high-efficiency trend due to partitions being a simpler decision-target compared to coarsening's neighbors. Overall, our average efficiency over time always exceeds 70%, and optimizing for peak efficiency on small late-stage hypergraphs would have a low impact on the end-to-end time. These observations further explain and corroborate our previous conclusions from the roofline mode.

Concerning memory utilization, Fig 14 presents the amount of memory occupied by different data structures throughout levels. The baseline memory occupation throughout execution is that of h-edges and inbound sets, with an amount linear in $|N| \cdot d$. Then, as expected, neighborhoods initially occupy the majority of memory, proportionally to $|N| \cdot d \cdot h$, but that quickly fades after three to four coarsening levels, as they are deduplicated. In later refinement levels, events too allocate a significant amount of memory, in part due to the support arrays required for their manipulation. Ultimately, the sole limiting factor to our implementation running under limited VRAM is neighborhoods, one that is easily circumvented by running coarsening kernels in batches over nodes and keeping only a few nodes' neighborhoods in device memory at once.

We report our scalability across GPUs in Fig. 10. There, we see how execution time grows with a similar linear trend regardless of the hardware. Additionally, the speedup between the `A100` and `GH200` increases from 1.26× on the three smallest h-graphs up to 1.85× on larger ones. Small h-graphs underutilize both GPUs, showing only some benefit from architectural improvements. At scale, however, the speedup is higher than the streaming multiprocessors increase $^{144}/_{108} \approx 1.33\times$ between the two GPUs. This suggests that, not only is the extra parallelism fully exploited, but also other hardware advantages benefit performance, namely memory bandwidth, cache hierarchy, and scheduling efficiency. Overall, these results indicate robust scalability with problem size and consistent performance gains across GPU generations.

### D. Ablation Studies

Our ablation studies are reported in Fig. 16. In the first case, we disabled our neighbors materialization, causing the spillage of histograms and deduplication to global mem-

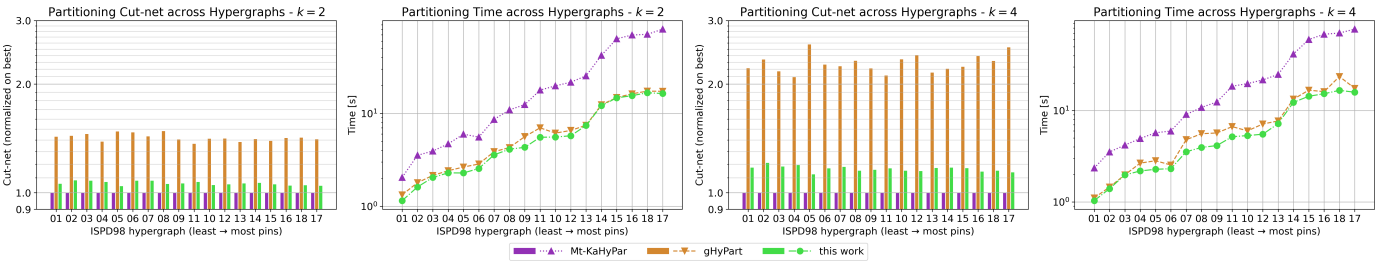



Fig. 15: Balanced $k$-way partitioning results comparison with two parallel methods on the ISPD98 h-graphs. Constraints: $k = 2, 4$, $\epsilon = 0.03$.

| | $o_1$ | $o_2$ | $o_3$ | $o_4$ | $o_5$ | $o_6$ | $o_7$ | $o_8$ | $o_9$ | $o_{10}$ | $o_{11}$ | $o_{12}$ | $o_{13}$ | $o_{14}$ | $o_{15}$ | $o_{16}$ | $o_{17}$ | $o_{18}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\|N\|$ | 204k | 314k | 370k | 440k | 470k | 520k | 735k | 821k | 854k | 1.11M | 1.13M | 1.14M | 1.35M | 2.36M | 2.59M | 2.94M | 2.97M | 3.37M |
| $\sum_{e\in E}\|e\|$ | 808k | 1.30M | 1.49M | 1.69M | 2.03M | 2.05M | 2.81M | 3.29M | 3.56M | 4.76M | 4.49M | 5.09M | 5.72M | 8.75M | 11.45M | 12.46M | 13.76M | 13.12M |
| $\text{avg}_{e\in E}\|e\|$ | 3.58 | 4.15 | 3.41 | 3.31 | 4.46 | 3.68 | 3.65 | 4.07 | 3.65 | 3.96 | 3.45 | 4.12 | 3.58 | 3.58 | 3.84 | 4.10 | 4.54 | 4.06 |

TABLE III: ISPD98 hypergraphs scaled by 16× used in the experiments [21].

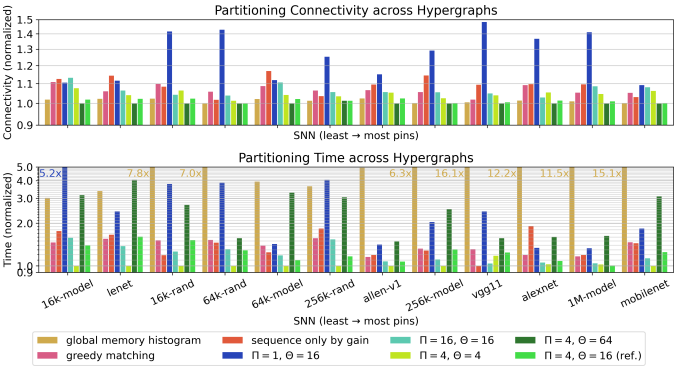



Fig. 16: Ablation study results across twelve SNNs hypergraphs.

ory. With candidates proposal being our costliest kernel, the impact of such a change more than tripled execution time, reinforcing our choice for materialization. Secondly, we swapped out our optimal matching for the greedy heuristic in [22], which always forces roots into a pair and propagates decisions backwards from them. We thus see that an optimal matching yields an average 0.96× lower connectivity through better coarsening and initial solution, with also a minor speedup thanks to more pairs being constructed, leading to fewer coarsening levels. At last, we removed our refinement moves chaining step, merely constructing the sequence by gain [12]. This reduced the effectiveness of refinement under tight constraints and strains events generation, giving both a 1.1× higher connectivity and 1.16× the execution time. Hence, our decision in favor of chain-based ordering. Throughout these experiments we kept $\Pi = 4$ and $\Theta = 16$.

Continuing, in terms of our parameters, increasing the number of candidates $\Pi$ from 1 to 4 (under $\Theta = 16$) reduces required coarsening levels by 2-5, lowering mean time by 0.5× and connectivity by 0.8×. In particular, the average fraction of matched nodes per level grows from 76% to 94% while initial partitions count drops to 0.84×, suggesting that $\Pi = 1$ is too conservative to coarsen effectively. Going further, to $\Pi = 16$, we see the opposite occur, nearly all nodes are matched per level, time improves by a marginal 0.98×, but the exceedingly aggressive coarsening hurts connectivity by 1.05×. All other alternatives for $\Pi \in \{1, \dots 16\}$ were also examined, but 4 offered the best quality-speed balance.

A higher number of refinement repetitions per level $\Theta$, going from 4 to 16 (under $\Pi = 4$), lowers connectivity by an average 0.94×, at the cost of 1.21× the execution time. Reaching for $\Theta = 64$, overall connectivity drops by another 0.98× for a 1.96× longer execution. Once more, after sweeping every $\Theta \in \{1, \dots 64\}$, we determined that 16 yielded the highest return on the spent time. Moreover, under these conditions, the amount of time dedicated to refinement almost matches that of coarsening.

Different configurations for noise threshold and chaining grade parameters (e.g. $\alpha$, $\beta$) were also evaluated, but with negligible effects.

We also considered the use of CUDA Graphs to reduce kernel launch overhead. However, due to the data-dependent control flow and constantly changing memory layouts across coarsening levels, graph replay provided limited benefit (1.02× speedup), and was therefore not adopted.

### E. Validation on k-way Partitioning

In Fig. 15 we report the comparison results for $k$-way balanced partitioning. Starting from $k = 2$, $\epsilon = 0.03$. In terms of speedup over Mt-KaHyPar, our implementation is 1.8× faster on smaller h-graphs, and progressively grows to a consistent 5× on the four larger ones. We are instead always 1.0-1.3× faster than gHyPart. Partitioning quality is dominated by Mt-KaHyPar, but our method lags a solid 1.05× behind on average cut-net. This is to be expected, given the variety of heuristics available on CPU that are incompatible with GPU parallelism, most notably the use of localized backtracking. Nonetheless, reaching such close results confirms the effectiveness of our algorithms. On the other hand, our approach consistently improves on gHyPart, with an average 0.75× cut-net. On $k = 4$, same $\epsilon$, we observe a similar pattern in execution time, while cut-net gaps widen slightly to a mean 1.16× between Mt-KaHyPar and our method, and 0.51× between gHyPart and us.

From the above, our solution also proves competitive on $k$-way partitioning. It achieves near-optimal results in spite of the cut-net not being its primary optimization metric and the burden of extra constraint checks. At the same time, execution time remains closely aligned with that of an existing GPU partitioner, while substantially improving on its cut-net quality. These observations testify to the soundness of our parallel implementation of the multi-level scheme.

## VIII. Conclusion

In this article, we presented the first GPU-centric design and implementation of a deterministic multi-level hypergraph partitioner that supports both partition size and distinct inbound h-edge constraints. In light of our experiments, it currently achieves among the best results in the field, scaling to hypergraphs with billions of pins with a time-to-solution of tens of seconds. In fact, execution time grows linearly with the number of pins, while maintaining strong cross-device scalability and efficient GPU resource utilization. The additional complexity introduced by our particular constraints was subdued with minimal overhead. Moreover, all major algorithms we proposed in this work have been shown to contribute to the final quality of results; their GPU-friendly design being reflected by the efficient use of hierarchical parallelism. At last, we demonstrated the generalizability of our solution to $k$-way partitioning, with a compelling balance of speed and results worth. Hence, advancing support for different constraint sets and optimization metrics constitutes a valuable line of future research. As of this publication, our implementation is available as open-source [28].

## Acknowledgements

The authors thank Prof. Sebastiano Schifano and Prof. Cristian Zambelli (Università degli Studi di Ferrara) for providing access to their computing infrastructure, which was used to conduct part of the experiments presented in this work.

This work has been partially supported by the Spoke 1 on *Future HPC* of the Italian Research Center on High-Performance Computing, Big Data and Quantum Computing (ICSC) funded by MUR Mission 4 - Next Generation EU.

## References

[1] U. Çatalyürek, K. Devine, M. Faraj, L. Gottesbüren, T. Heuer, H. Meyerhenke, P. Sanders, S. Schlag, C. Schulz, D. Seemaier, and D. Wagner, "More recent advances in (hyper)graph partitioning," *ACM Comput. Surv.*, vol. 55, no. 12, Mar. 2023. [Online]. Available: https://doi.org/10.1145/3571808

[2] P. A. Papp, G. Anegg, and A.-J. N. Yzelman, "Partitioning hypergraphs is hard: Models, inapproximability, and applications," in *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 415–425. [Online]. Available: https://doi.org/10.1145/3558481.3591087

[3] Z. Wu, H. Zhao, H. Liu, W. Wen, and J. Li, "ghypart: Gpu-friendly end-to-end hypergraph partitioner," *ACM Trans. Archit. Code Optim.*, vol. 22, no. 1, Mar. 2025. [Online]. Available: https://doi.org/10.1145/3711925

[4] M. Ronzani and C. Silvano, "A case for hypergraphs to model and map snns on neuromorphic hardware," 2026. [Online]. Available: https://arxiv.org/abs/2601.16118

[5] O. Jin, Q. Xing, Y. Li, S. Deng, S. He, and G. Pan, "Mapping very large scale spiking neuron network to neuromorphic hardware," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 419–432. [Online]. Available: https://doi.org/10.1145/3582016.3582038

[6] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, "Multilevel hypergraph partitioning: Applications in vlsi domain," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 1, pp. 69–79, 1999.

[7] F. Li, Y. Wang, M. Lu, Y. Zhu, H. Wang, Z. Zhao, J. Huang, X. Wei, X. Liang, Y. Wang, H. Xu, H. Li, X. Li, Q. Liu, M. Liu, N. Sun, and Y. Han, "The decomposition and combination paradigms of chiplet-based integrated chips," *Integrated Circuits and Systems*, vol. 1, no. 1, pp. 18–30, 2024.

[8] K. D. Devine, E. G. Boman, R. T. Heaphy, B. A. Hendrickson, J. D. Teresco, J. Faik, J. E. Flaherty, and L. G. Gervasio, "New challenges in dynamic load balancing," *Applied Numerical Mathematics*, vol. 52, no. 2, pp. 133–152, 2005, aDAPT '03: Conference on Adaptive Methods for Partial Differential Equations and Large-Scale Computation. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0168927404001631

[9] G. Ballard, A. Druinsky, N. Knight, and O. Schwartz, "Hypergraph partitioning for sparse matrix-matrix multiplication," *ACM Trans. Parallel Comput.*, vol. 3, no. 3, Dec. 2016. [Online]. Available: https://doi.org/10.1145/3015144

[10] L. Cheng, H. Cho, and P. Yoon, "An accelerated procedure for hypergraph coarsening on the gpu," in *2015 IEEE High Performance Extreme Computing Conference (HPEC)*, 2015, pp. 1–7.

[11] F. Khorasani, R. Gupta, and L. N. Bhuyan, "Scalable simd-efficient graph processing on gpus," in *2015 International Conference on Parallel Architecture and Compilation (PACT)*, 2015, pp. 39–50.

[12] W. L. Lee, D.-L. Lin, C.-H. Chiu, U. Schlichtmann, and T.-W. Huang, "Hyperg: Multilevel gpu-accelerated k-way hypergraph partitioner," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1031–1040. [Online]. Available: https://doi.org/10.1145/3658617.3697551

[13] G. Karypis and V. Kumar, "Multilevel k-way hypergraph partitioning," in *Proceedings of the 36th Annual ACM/IEEE Design Automation Conference*, ser. DAC '99. New York, NY, USA: Association for Computing Machinery, 1999, p. 343–348. [Online]. Available: https://doi.org/10.1145/309847.309954

[14] S. Schlag, T. Heuer, L. Gottesbüren, Y. Akhremtsev, C. Schulz, and P. Sanders, "High-quality hypergraph partitioning," *ACM J. Exp. Algorithmics*, vol. 27, Feb. 2023. [Online]. Available: https://doi.org/10.1145/3529090

[15] U. Catalyurek and C. Aykanat, "Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 10, no. 7, pp. 673–693, 1999.

[16] L. Gottesbüren, T. Heuer, N. Maas, P. Sanders, and S. Schlag, "Scalable high-quality hypergraph partitioning," *ACM Trans. Algorithms*, vol. 20, no. 1, Jan. 2024. [Online]. Available: https://doi.org/10.1145/3626527

[17] S. Maleki, U. Agarwal, M. Burtscher, and K. Pingali, "Bipart: a parallel and deterministic hypergraph partitioner," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 161–174. [Online]. Available: https://doi.org/10.1145/3437801.3441611

[18] K. Devine, E. Boman, R. Heaphy, R. Bisseling, and U. Catalyurek, "Parallel hypergraph partitioning for scientific computing," in *Proceedings 20th IEEE International Parallel & Distributed Processing Symposium*, 2006.

[19] W. L. Lee, D.-L. Lin, T.-W. Huang, S. Jiang, T.-Y. Ho, Y. Lin, and B. Yu, "G-kway: Multilevel gpu-accelerated k-way graph partitioner," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3649329.3656238

[20] C. Fiduccia and R. Mattheyses, "A linear-time heuristic for improving network partitions," in *19th Design Automation Conference*, 1982, pp. 175–181.

[21] C. J. Alpert, "The ispd98 circuit benchmark suite," in *Proceedings of the 1998 International Symposium on Physical Design*, ser. ISPD '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 80–85. [Online]. Available: https://doi.org/10.1145/274535.274546

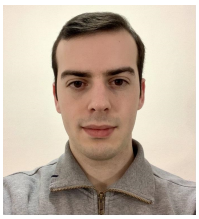
[22] M. Ronzani and C. Silvano, "Incidence constraints in hypergraph partitioning on gpu," 2026, accepted at AsHES Workshop @ IPDPS 2026. [Online]. Available: https://arxiv.org/abs/2604.14411

[23] M. Cygan, F. V. Fomin, D. Marx, S. Saurabh, L. Kowalik, D. Lokshtanov, and M. Pilipczuk, *Parameterized Algorithms*, 1st ed. Cham, Switzerland: Springer International Publishing, Jul. 2015.

[24] C. Gupta, R. Latypov, Y. Maus, S. Pai, S. Särkkä, J. Studený, J. Suomela, J. Uitto, and H. Vahidi, "Fast dynamic programming in trees in the mpc model," in *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 443–453. [Online]. Available: https://doi.org/10.1145/3558481.3591098

[25] M. Ronzani, "Spiking neural network hypergraphs with spike frequency data," Mar. 2026. [Online]. Available: https://doi.org/10.5281/zenodo.19194881

[26] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1409.1556
[27] Y. N. Billeh *et al.*, "Systematic integration of structural and functional data into multi-scale models of mouse primary visual cortex," *Neuron*, vol. 106, no. 3, pp. 388–403.e18, May 2020. [Online]. Available: https://doi.org/10.1016/j.neuron.2020.01.040
[28] M. Ronzani, "open-source artifact," https://github.com/EMJzero/AxonCUDA, 2026, accessed: 2026-04-01.



**Marco Ronzani** received the B.S. and M.S. degrees in Computer Science and Engineering from Politecnico di Milano, Italy, in 2022 and 2024, respectively. He is currently pursuing the Ph.D. degree at the same institution. His research interests include algorithm engineering for parallel and GPU computing, with a focus on the runtime optimization of hardware accelerators through large-scale combinatorial methods. His current work explores hypergraph algorithms for neuromorphic computing systems.



**Cristina Silvano** is a Full Professor of Computer Architecture at Politecnico di Milano, where she is the Chair of the Research Area on Computer Science and Engineering. In 2022, she was one of the promoters of the new M.Sc. degree in HPC Engineering at Politecnico di Milano. Since 2017, she is an IEEE Fellow for contributions to energy-efficient computer architectures. Her research activities are in the areas of computer architecture and EDA, with emphasis on design space exploration of energy-efficient architectures, low-power design of manycore architectures, accelerators for deep neural networks, and application autotuning for HPC. She has published more than 200 peer-reviewed papers, six books, and some patents. She has been Scientific Coordinator of three European research projects (ANTAREX, 2PARMA and MULTICUBE). She is an active member of the scientific community and she regularly serves in several international program committees. She is Associate Editor of the ACM Trans. on Computer Architecture and Compiler Optimization and Associate Editor-in-Chief of the Journal on Parallel and Distributed Computing. She serves regularly as independent expert reviewer for the European Commission and for several national science foundations. In 2022, she was in the National Expert Group on Semiconductor Technologies appointed by the Italian Ministry of University and Research.